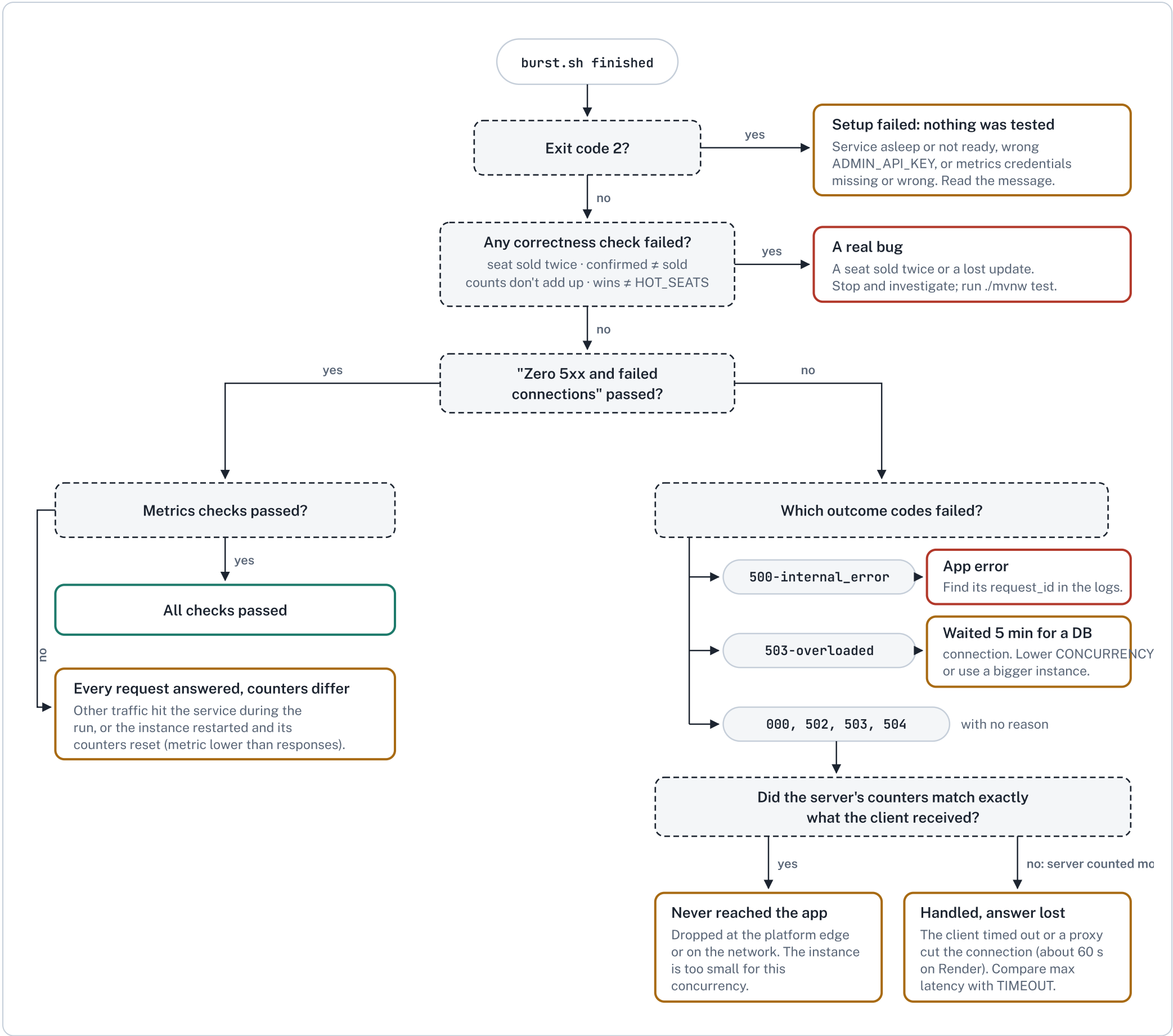


SEAT_SELECTION · BURST.SH

Burst Failure Guide

When `burst.sh` reports a failure, which checks failed, together with the `outcomes: lines`, usually tells you what actually happened. Start at the top and follow your run's answers down.



question about your run all good a problem in the service's code setup, capacity or network: the service's decisions were still correct

Correctness checks come first: if one fails, nothing else matters until it's explained. Delivery failures are separated by comparing the server's own counters with what the client received. If they match exactly, the failed requests never reached the app; if the server counted more, it answered but the answer was lost on the way back.

Worked examples from the live service

RUN	WHAT FAILED	PATH THROUGH THE CHART
2,250 requests, 300 in flight	1 retry 000; the server counted 950 replays, the client received 949; max latency 61 s	Correctness passed → 5xx failed → 000 → server counted more → handled, answer lost : the connection was cut at about 60 s.
18,500 requests, 300 in flight	37 failed requests; every metric matched the responses exactly	Correctness passed → 5xx failed → no reason given → counters matched → never reached the app : the free instance couldn't take 300 in flight.

Reading the other lines

- **Scenario counts come up short together with delivery failures.** The hot-seat, retry and stampede tallies drop by the number of failed requests. As long as wins = HOT_SEATS still holds, the scenario logic is fine.
- **"Timed out" invariant snapshots aren't failures.** A seat-map poll that didn't answer in time proves nothing either way. Only snapshots whose counts don't add up fail the check.
- **A metric higher than the responses:** the app counted something the client never saw. **A metric lower than the responses:** the counters were reset during the run, usually by a restart.